Introduction to Complexity

CS300: Programming II

Cole Nelson

Learning Objectives

1. Be able to **describe** how a method's step count grows when N doubles.
2. Be able to **classify** a method as O(1), O(N), or O(N^2).
3. Be able to **identify** the sizes a method's steps depend on.

Will it still work?

```
public static boolean hasDuplicateTitles(String[] titles) {
  for (int i = 0; i < titles.length; i++) {
    for (int j = i + 1; j < titles.length; j++) {
      if (titles[i].equals(titles[j])) {
        return true;
      }
    }
  }
  return false;
}
```

Instant on our 4 songs. A real library has 100,000...

Songs	Time
10,000	0.1 seconds
25,000	0.5 seconds
50,000	2 seconds
100,000	25 seconds
200,000	over 2 minutes

Why not just time it? Times change from laptop to laptop, and run to run. **Count the steps instead.**

Two ways to the same answer

```
// Add up 1 + 2 + ... + n, one at a time
int total = 0;
for (int i = 1; i <= n; i++) {
  total += i;
}

// Or all at once
int total2 = n * (n + 1) / 2;
```

Not all algorithms are created equal. The loop takes **n steps**, one per pass. The formula takes **1**.

Eclipse Exercise

Get today's files from Canvas, and open `TotalSeconds.java`.

Run it. Double the songs, and the steps...?

Double the songs, double the steps

Songs	`totalSeconds`	steps
1,000	1,000	
2,000	2,000	
4,000	4,000	

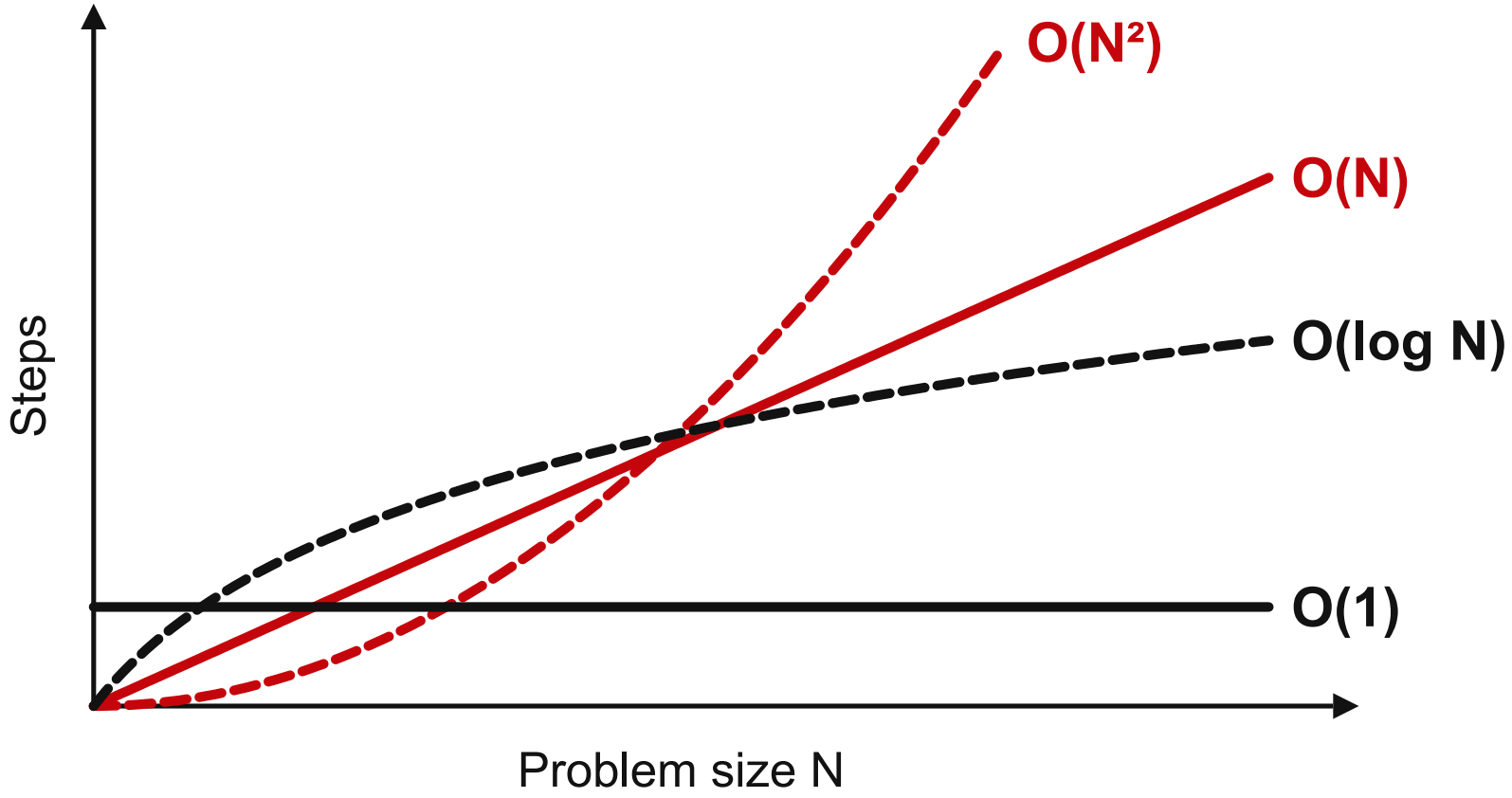
N songs, N steps. We write that as **O(N)**: "big O of N".

Big-O notation

Definition: Big-O notation describes how the steps grow as N grows: the *shape*, not the exact count.

```
5 steps           ->  O(1)
2N + 3 steps      ->  O(N)      drop the 2 and the 3
3N^2 + 2N steps   ->  O(N^2)    keep N^2: it grows fastest
```

A big part of Big-O is generalizing and approximating!


O(N²)
O(N)
O(log N)
O(1)
Steps
Problem size N

Everyday tasks

- Find the first name on a class list? **O(1)**.
- Read every name on it? **O(N)**.
- Look up a word in a paper dictionary? **O(log N)**: open to the middle, keep the half it's in.
- Do any two students share a birthday? **O(N²)**: check every pair.

Tophat Q1

For `n = 1000`, how many steps? What's the Big-O?

```
for (int i = 0; i < n; i += 2) {
  steps++;
}
```

→ Open Tophat to vote.

Eclipse Exercise

Open `HasDuplicates.java`.

TODO 1a. 1,000 songs take 499,500 steps. Guess 2,000, and the Big-O. Then run it.

TODO 1b. Change `makeLibrary` to `makeRandomLibrary`. Run it twice.

Double the songs, ×4 the steps

Songs	`hasDuplicateTitles` steps
1,000	499,500
2,000	1,999,000
4,000	7,998,000

Each song is compared with every song after it: about N × N / 2 steps. That's **O(N²)**. 200,000 songs? **20 billion steps.** That's 2 minutes+.

The steps go ×4 every time. The times didn't!

What if titles repeat?

`makeRandomLibrary` makes titles like *Lonely Blue River*. In a big library, two songs almost always share a title, like birthdays in a big class.

Why so few steps, and different every run? **It stops at the first repeat.**

Big-O counts the **worst case**: no repeats, so it checks every pair. Still **O(N²)**. More on Friday.

Tophat Q2

What Big-O do these steps suggest?

Songs	Steps
1,000	3,000
2,000	12,000
3,000	27,000

→ Open Tophat to vote.

Eclipse Exercise

Open `CountLetters.java`. Two loops, so $O(N^2)$?

TODO 2. 1,000 titles take 7,893 steps. Guess 2,000, then run it. Then think: what if every title were twice as long?

Count every letter in every title

Titles	`countLetters` steps
1,000	7,893
2,000	16,893
4,000	34,893

Two loops, so O(N²)? **No!** Double the titles, and the steps about double. (A bit more: `Song 1000` has more letters than `Song 999`.)

Two loops, two sizes

```
int count = 0;
for (int i = 0; i < library.length; i++) {           // N titles
  for (int j = 0; j < library[i].length(); j++) {    // M letters each
    count++;
  }
}
```

M letters, N times: M × N steps. Double either one, and the steps double: **O(M × N)**. If M never changes, it's just O(N).

O(N²) is when both loops go over the same N, like `hasDuplicateTitles`.

Tophat Q3

What's the Big-O?

```
for (int i = 0; i < n; i++) {
  for (int j = 0; j < 3; j++) {
    steps++;
  }
}
```

→ Open Tophat to vote.

Tophat Q4

Titles get 10 times longer. What happens to the steps?

```
for (int i = 0; i < library.length; i++) {
  System.out.println(library[i].charAt(0));
}
```

→ Open Tophat to vote.

Questions?